

# Informe Tecnico EFT - MiniMarket Plus

Asignatura: Desarrollo Backend II (PBY2202). Caso: cadena de minimarkets MiniMarket Plus. La solucion entrega un backend Spring Boot con dominios separados para autenticacion, inventario, pedidos/promociones y reportes, aplicando seguridad JWT, roles, pruebas, OpenAPI y HATEOAS.

## 1. Arquitectura y microservicios

La solucion se organiza por dominios de negocio, manteniendo bajo acoplamiento y responsabilidades claras. Cada paquete representa un microservicio funcional dentro del backend evaluable: auth, inventory, orders y reports.

| Componente | Responsabilidad                                                                  | Endpoints principales                                                  |
|------------|----------------------------------------------------------------------------------|------------------------------------------------------------------------|
| auth       | Login, emision de JWT y usuarios por rol.                                        | /api/auth/login                                                        |
| inventory  | Stock centralizado por sucursal, disponibilidad y ordenes de compra automaticos. | /api/products, /api/inventory/products, /api/inventory/purchase-orders |
| orders     | Pedidos online, retiro/despacho y promociones centralizadas.                     | /api/orders, /api/promotions                                           |
| reports    | Reportes de ventas y rotacion de productos.                                      | /api/reports/sales, /api/reports/rotation                              |

## 2. Seguridad aplicada

Se implementa Spring Security en modo stateless con filtro JWT. Los usuarios de demostracion tienen claves cifradas con BCrypt y roles ADMIN, CAJERO y CLIENTE. Los reportes y operaciones de inventario requieren ADMIN; los pedidos aceptan CLIENTE, CAJERO o ADMIN; productos y promociones quedan publicos para consulta.

Credenciales de prueba: admin/admin123, cajero/cajero123 y cliente/cliente123.

## 3. Funcionalidades implementadas

Inventario: productos por sucursal, stock minimo, actualizacion de stock y generacion automatica de ordenes de compra cuando el stock llega al minimo.

Ventas y pedidos: creacion de pedidos para retiro o despacho, validacion de stock, descuento automatico del inventario y promociones por categoria.

Reportes: rotacion de productos ordenada por unidades vendidas y resumen consolidado de ventas.

## 4. Pruebas unitarias y evidencia

Se ejecuto el comando mvn -Dmaven.repo.local=target/.m2 test. Resultado: Tests run: 6, Failures: 0, Errors: 0, Skipped: 0. Las pruebas cubren creacion y lectura de JWT, reglas de stock, ordenes de compra automaticas, creacion de pedido con descuento de inventario y proteccion de reportes administrativos.

| Prueba         | Validacion                            |
|----------------|---------------------------------------|
| JwtServiceTest | El token incluye usuario y rol ADMIN. |

|                         |                                                                  |
|-------------------------|------------------------------------------------------------------|
| InventoryServiceTest    | Genera orden de compra con stock bajo y rechaza venta sin stock. |
| OrderServiceTest        | Crea pedido confirmado, aplica promocion y descuenta stock.      |
| SecurityIntegrationTest | Login retorna JWT y reportes administrativos exigen token ADMIN. |

## 5. OpenAPI y HATEOAS

La API se documenta automaticamente con springdoc en /swagger-ui.html y /v3/api-docs. Adicionalmente se entrega docs/openapi.yaml como especificacion OAS. Las respuestas de productos, pedidos y reportes incluyen enlaces \_links para navegacion HATEOAS.

## 6. Mejoras aplicadas

Se aplicaron validaciones de entrada, manejo centralizado de errores, seguridad por roles, separacion de responsabilidades, datos de demostracion para ejecucion inmediata, pruebas automatizadas y documentacion tecnica de uso.

## 7. Evidencia paso a paso sugerida

1. Ejecutar mvn test. 2. Levantar la API con mvn spring-boot:run. 3. Login como admin y cliente. 4. Consultar productos publicos. 5. Crear pedido y verificar descuento de stock. 6. Actualizar stock bajo minimo y revisar ordenes de compra. 7. Abrir Swagger UI y mostrar enlaces HATEOAS en respuestas JSON.

## 8. Conclusiones

La solucion permite centralizar inventario, habilitar pedidos digitales y proteger operaciones criticas mediante JWT y roles. La estructura facilita mantenimiento y futuras expansiones, por ejemplo persistencia con base de datos, despliegue por contenedores y separacion fisica de servicios.